

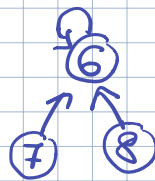
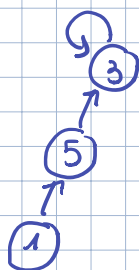
Insiemi Disgiunti Implementazione con Alberi



x.key
x.parent

Esempio

{ {1, 3, 5} , {2, 4} , {6, 7, 8} }



Make(x) {

% x è un oggetto di tipo nodo

% x.key contiene già il valore

x.parent ← x

return x

}

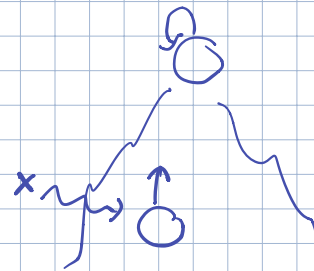


Rappresentante di un albero è la radice

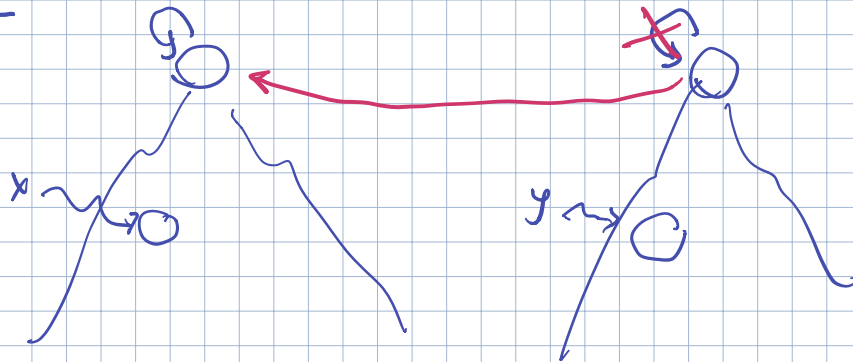
```

Find(x) {
  if (x ≠ x.parent) {
    return (Find(x.parent))
  } else {
    return x
  }
}

```



Union



```

Union(x, y) {
  z ← Find(x)
  w ← Find(y)
  if (z ≠ w) {
    Link(z, w)
  }
}

```

Link (E, w) if $\% E, w$ sono due radici

$w.parent \leftarrow E$

}

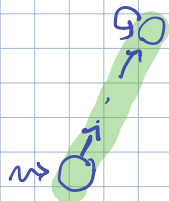
Costi

n costo di Make

m costo Make Union Find

1 Make $\mathcal{O}(1)$

1 Find $\mathcal{O}(n)$ $\mathcal{O}(h)$



1 Union $\mathcal{O}(n)$ (le due Find) $\mathcal{O}(h)$

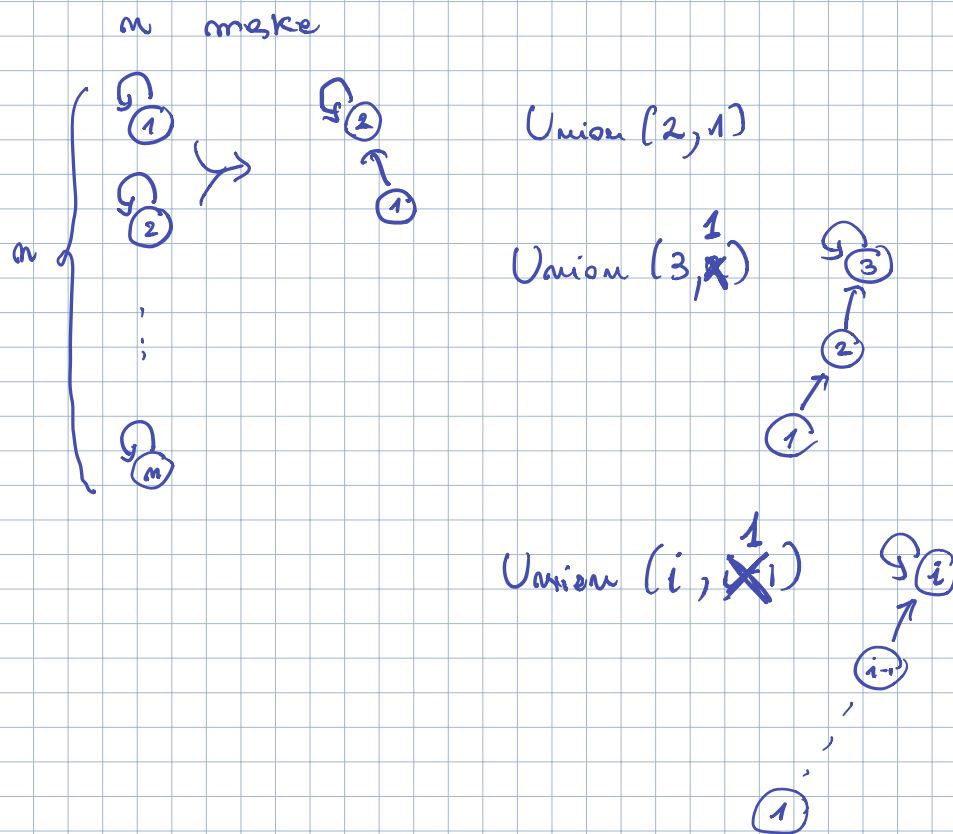
Complessivamente

$$n \cdot \mathcal{O}(1) + f \cdot \mathcal{O}(n) + u \cdot \mathcal{O}(n)$$

$$\leq \mathcal{O}(n) + m \cdot \mathcal{O}(n) + (n-1) \cdot \mathcal{O}(n)$$

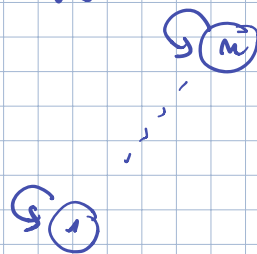
$$= \mathcal{O}(n \cdot m)$$

Si aggiunge la complessità $\mathcal{O}(n \cdot m)$ nel caso peggiore?



le i -esima union ~~$u(i)$~~
 $u(i)$

Atte fine ho



eseguo f Find(1)

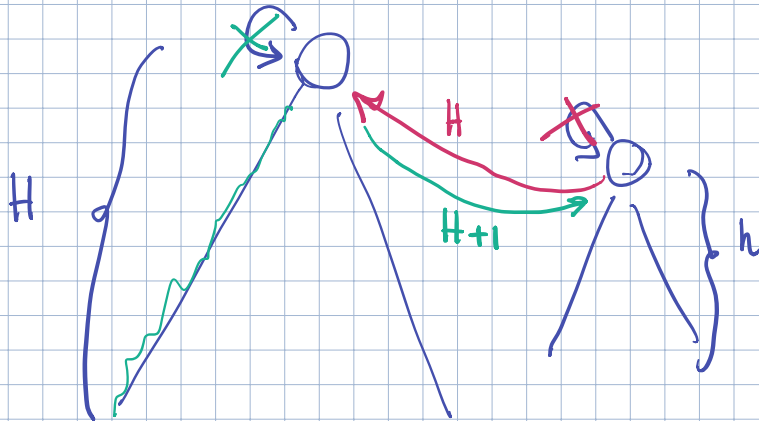
$$u(n) + \sum_{i=2}^n u(i) + f \cdot u(n) =$$

$$= u(n) + u(n^2) + f \cdot u(n) = u(n \cdot n)$$

$f = u(n)$

Come miglioriamo il costo?

Gli alberi non devono bilanciarsi con le Union



Devo unire gli alberi in modo da far crescere l'altezza solo quando è necessario

- Devo aggiungere l'albero più basso e quello più alto
- L'altezza cresce solo quando unisco due alberi aventi la stessa altezza

Union by rank

rank $\propto$ altezza

Devo memorizzare il rango dell'albero

In ogni nodo aggiungo un campo $x.rank$ che indica il rango (altezza) dell'albero radicato in x

```
Make(x) {  
    x.parent ← x  
    x.rank ← 0  
}
```



Find(x) invertito

```
Union-by-rank(x, y) {  
    z ← Find(x)  
    w ← Find(y)  
    if (z ≠ w) {  
        if (z.rank > w.rank) {  
            Link(z, w)  
        } else {  
            Link(w, z)  
            if (w.rank = z.rank) {  
                w.rank ← w.rank + 1  
            }  
        }  
    }  
}
```

Esempio

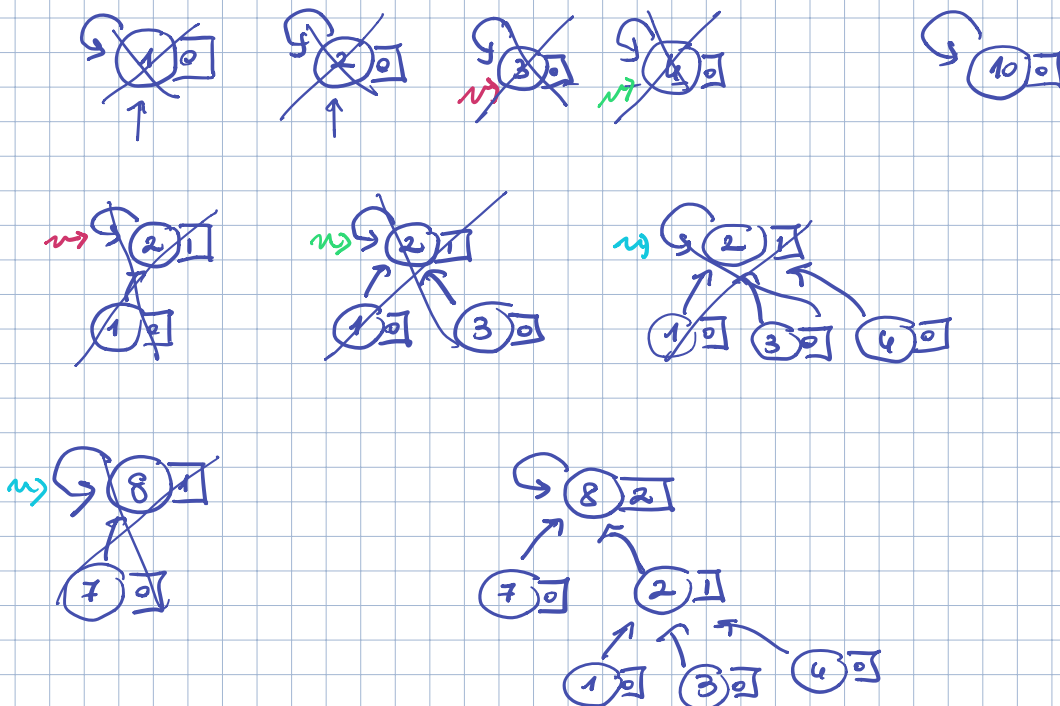
Costruire gli insiemi disgiunti con dischi
1, 2, ..., 10

Esegui $\text{Union}(2,1)$ $\text{Union}(3,1)$

$\text{Union}(4,1)$, $\text{Union}(7,8)$, $\text{Union}(1,8)$

Utilizzando Union by Rank

Nel caso di alberi aventi lo stesso rango
si utilizza come radice quello con dischi
maggiore



Complessità

Quale è la complessità nel caso peggiore?

1 Make $\Theta(1)$

1 Find $O(h)$

1 Union-by-rank $O(h)$

Complessivamente

$$m \cdot \Theta(1) + f \cdot O(h) + u \cdot O(h) =$$

$$\Theta(m) + O(m) \cdot O(h)$$

$\uparrow$
 $m = m + f + u$

Lemma

Utilizzando Union-by-rank a partire da m operazioni make si ottengono alberi aventi altezza $O(\log m)$

Dim.

Per induzione su h

5) Un albero di altezza h costruito con Union by Rank contiene almeno 2^h nodi

BASE $h=0$



$$2^0 = 1$$

✓

PASSO

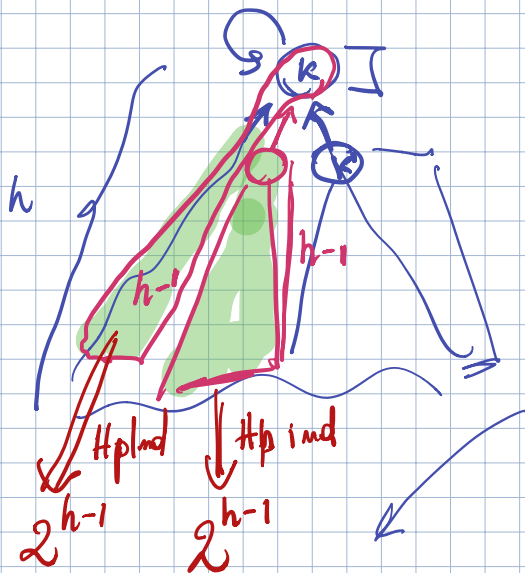
Hplnd)

Un albero di altezza $l < h$

almeno 2^l nodi

II)

h almeno 2^h nodi



Union (k, k')

Avendo unito in uno
dei passi precedenti
due alberi di altezza
 $h-1$ ottenendo un
albero di altezza h

$$\text{almeno } 2^{h-1} + 2^{h-1} = 2 \cdot 2^{h-1} = 2^h$$

Costi

$O(1)$

Make

$O(\log n)$

Find

$O(\log n)$

Union-by-Rank

Complessiv.

$$O(1) + O(m \cdot \log n)$$

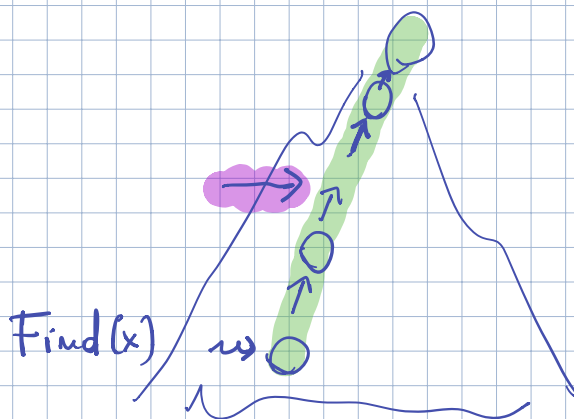
$$= O(m \cdot \log n)$$

con Union-by-rank

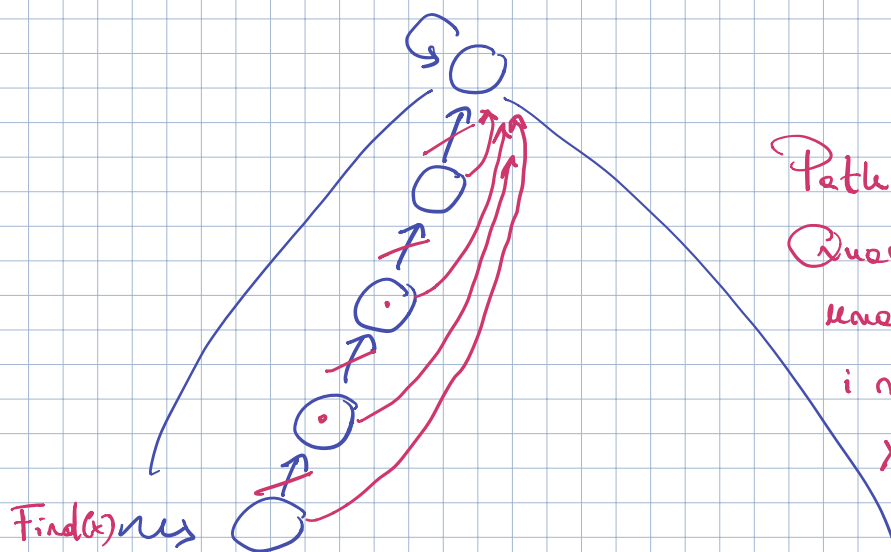
Domanda

le rango coincide con l'altezza?

~ ~ ~



Se eseguo tante
volte Find(x)
ogni volta ho costo
 $O(\log n)$
e ogni volta ripercorro
lo stesso ramo

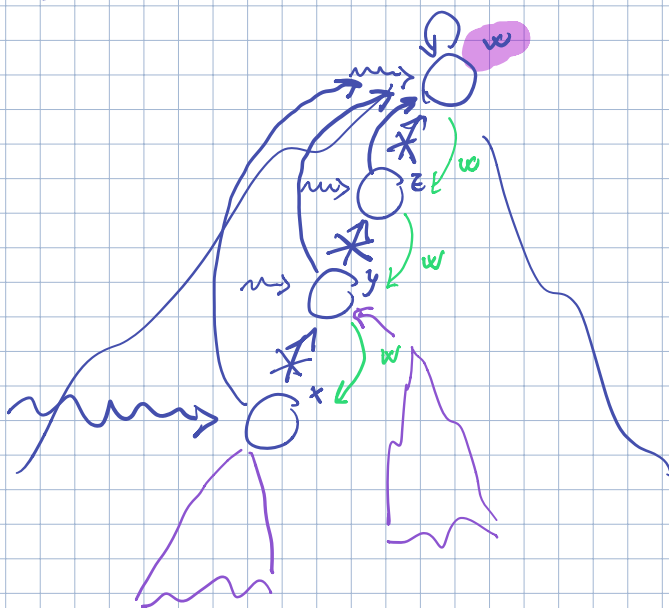


Path Compression
Quando eseguo
una Find tutti
i nodi sul cam.
X-radice vengono
aggianciati alla
radice

```

Find_PC(x) {
  if (x ≠ x.parent) {
    x.parent ← Find_PC(x.parent)
  } else {
    return x
  }
}

```



Comprimmo
solo il cammino
x-radice

Vedere Costi !!